Disciplina: Sistemas Distribuídos
Equipe: Felipe Henriques Batista do Nascimento, João Arthur Britto, Raíssa Quezia

TDE I - Sistema de Cardápio Digital e Pedidos para Rede de Lanchonetes em Feira de Santana

Apresentação da Ideia

O **Sistema Distribuído de Cardápio Digital e Pedidos** foi desenvolvido para resolver um desafio crítico de arquiteturas modernas na Web: a eficiência de tráfego de dados e a consistência de atualizações em tempo real (como o fluxo entre o cliente acessando o cardápio via navegador web e o painel KDS - *Kitchen Display System* da cozinha) em um cenário de alto volume de acessos em uma rede de lanchonetes em Feira de Santana.

Em aplicações corporativas com múltiplos pontos de atendimento e cardápios dinâmicos, o uso de APIs REST tradicionais frequentemente esbarra em problemas de *over-fetching* (tráfego de dados desnecessários que sobrecarregam o navegador) e *under-fetching* (múltiplas requisições HTTP encadeadas para montar uma única tela, como dados do produto, preço e ingredientes). O projeto aplica conceitos fundamentais de **Sistemas Distribuídos** utilizando **GraphQL** como motor unificado de requisições sobre HTTP/POST, permitindo que o front-end web estruture os dados que precisa em uma única ida e volta ao servidor.

Além disso, a arquitetura emprega **GraphQL Subscriptions** baseadas em protocolo WebSocket integradas a um broker **Redis (Pub/Sub)**, garantindo que alterações de status de pedidos (como a transição de `RECEBIDO` para `EM_PREPARO` e `PRONTO`) sejam refletidas instantaneamente e de forma descentralizada entre os nós da cozinha e a interface web do cliente. Para assegurar robustez, o microsserviço de pedidos utiliza **Idempotência baseada em chaves de transação (UUIDv4)** no PostgreSQL para evitar duplicação de comandas em caso de instabilidades na rede.

Na ponta operacional, a solução oferece interfaces web desenvolvidas em Next.js: o **Portal Web do Cliente** (para navegação no cardápio e submissão do pedido) e o **Painel KDS da Cozinha** (com atualizações via *Subscriptions* em tempo real). Toda a arquitetura foi planejada para operar de ponta a ponta tanto localmente via Docker Compose quanto preparada para nuvem.

Escopo do Projeto

Objetivo Geral

Implementar e demonstrar uma malha de comunicação distribuída flexível, fortemente tipada e otimizada por meio de um Servidor GraphQL (SDL), integrando interfaces web de alta performance e microsserviços de negócio (Catálogo e Pedidos) para processamento de cardápios e fluxo automatizado de cozinha em tempo real.

Escopo Principal

O foco do projeto está na camada de API, no contrato de interface e nas garantias de entrega de dados dinâmicos:

- **Contrato e Definição de Interface (IDL / SDL):**
  - Definição formal e canônica do esquema GraphQL contendo os tipos (`Produto`, `Pedido`), *Queries* de leitura otimizada e *Mutations* de escrita transacional.
  - Modelagem estrita de tipos para identificadores (`ID!`), valores monetários (`Float!`), listas restritas de ingredientes e estados de pedidos.
- **Motor de Consultas e Resolução (GraphQL sobre HTTP/POST):**
  - Resolução de consultas aninhadas (*Nested Queries*) permitindo que um único payload traga dados do produto e suas restrições/ingredientes em uma única requisição ao servidor, eliminando o *over-fetching*.
  - Tratamento de erros estruturados do GraphQL mantendo o padrão de resposta HTTP mesmo em falhas parciais de resolução de campos.
- **Comunicação Assíncrona em Tempo Real (GraphQL Subscriptions via WebSockets):**
  - Utilização de conexões persistentes WebSocket para transmissão de eventos assíncronos.
  - Integração com **Redis Pub/Sub** para desacoplar instâncias do servidor GraphQL e propagar eventos de mudança de status de pedidos de forma escalável entre os nós distribuídos.
- **Transparência de Acesso e Resiliência na Camada de Dados:**
  - Validação rigorosa de mutações para criação de pedidos com múltiplos itens.
  - Idempotência semântica e transacional no banco de dados para evitar duplicidade de comandas em caso de retentativas de envio pelo cliente web após oscilações de conexão.

Escopo Secundário

Estes componentes existem exclusivamente para validar, simular e dar suporte visual ao funcionamento do motor GraphQL:

- **Simulador de Clientes / Pedidos:** Mecanismo programático ou rotinas de teste para simular requisições concorrentes de clientes fazendo pedidos simultâneos no cardápio web.
- **Camada de Persistência:** Banco relacional PostgreSQL 16 para gerenciamento do catálogo de produtos e persistência transacional dos pedidos.
- **Interfaces Web para Demonstração:**
  - *Portal Web do Cliente (Porta 3000):* Navegação via browser pelo cardápio filtrado por categoria e montagem de carrinho.
  - *Painel KDS da Cozinha (Porta 3001):* Monitoramento via browser em tempo real dos pedidos recebidos com opções de avanço de status via *Mutations*.

- **Ambiente de Execução:** Orquestração integrada em contêineres via Docker Compose simulando a malha de microsserviços.

Critérios de Sucesso e Validação

- **Eliminação de Over-fetching:** Comprovar via ferramenta de rede do navegador web que a aplicação consome apenas os campos declarados na *Query* do GraphQL.
- **Respeito ao Schema (SDL):** Nenhuma requisição com tipagem ou campos inválidos deve ser processada pelo servidor.
- **Tempo Real Funcional (Subscriptions):** A alteração de status de um pedido na cozinha deve atualizar a tela web do cliente instantaneamente via WebSocket/Redis sem necessidade de F5 (*refresh*).
- **Resiliência e Idempotência:** Simular quedas de conexão do navegador durante o envio de um pedido e validar que o mecanismo de retransmissão não gera comandas duplicadas na base de dados.
- .

Estrutura do Projeto

```
sistemas-distribuidos-graphql/
├── schema/                                ---> [NÚCLEO CANÔNICO]
│   └── cardapio.graphql                   ---> Contrato IDL SDL (Types, Queries, Mutations e Subscriptions)
│
├── lanchonete-cliente-web/                ---> [NÓ CLIENTE WEB] (Portal do Cliente)
│   ├── src/
│   │   ├── graphql/                        ---> Stubs de requisição e operações
│   │   │   ├── queries.ts                  ---> Consultas otimizadas do cardápio (evita over-fetching)
│   │   │   └── mutations.ts                ---> Submissão transacional de novos pedidos
│   │   ├── subscriptions/                  ---> Camada de tempo real via WebSocket
│   │   │   └── order-status.sub.ts         ---> Escuta atualizações de status da cozinha em tempo real
│   │   └── app/                            ---> [Apoio] Interface web do cliente (Next.js 14)
│   │
├── lanchonete-api-gateway/                ---> [NÓ SERVIDOR CENTRAL / API GATEWAY]
│   ├── src/
│   │   ├── server/
│   │   │   ├── main.ts                     ---> Inicialização do Apollo Server (:4000) com suporte a WebSockets
│   │   │   ├── resolvers.ts                ---> Implementação dos resolvers de Query, Mutation e Subscription
│   │   │   └── idempotency.ts              ---> Validação de chaves de transação (UUIDv4) para evitar duplicidade
│   │   ├── redis/                          ---> Camada de mensageria Pub/Sub para escala distribuída
│   │   │   └── redis-pubsub.service.ts     ---> Propagação de eventos de pedidos entre nós
│   │   └── database/                       ---> Camada de persistência (Prisma ORM / PostgreSQL 16)
│   │       └── schema.prisma               ---> Mapeamento relacional de Produtos, Categorias e Pedidos
│   │
├── lanchonete-cozinha-kds/                  ---> [NÓ PAINEL DA COZINHA]
│   ├── src/
│   │   ├── graphql/
│   │   │   └── kds.operations.ts           ---> Subscriptions de novos pedidos e Mutations de avanço de status
│   │   └── frontend/                       ---> [Apoio] Painel KDS da Cozinha em tempo real (Next.js 14)
│   │
├── docker/                                ---> Configurações de redes e proxies de borda Nginx
├── terraform/                             ---> IaC para provisionamento de VPC, instâncias EC2 e RDS AWS
└── docker-compose.local.yml               ---> Orquestração integrada para execução de todo o ecossistema em 1 comando
```

Planejamento do Projeto

Fase 1: Concepção do Schema e Especificação IDL

- **Foco:** Definição da interface e modelagem do domínio distribuído.

- **Entregas:**
    - Criação do arquivo de definição de esquema (schema.graphql ou SDL) com os tipos Produto, Pedido, Query, Mutation e Subscription.
    - Configuração inicial do servidor Apollo Server com TypeScript/Node.js.

Fase 2: Implementação dos Microsserviços de Domínio

- **Foco:** Construção dos serviços de apoio (Catálogo e Pedidos).
- **Entregas:**
    - Desenvolvimento do microsserviço de **Catálogo** (responsável por gerenciar produtos, preços e categorias de lanches em Feira de Santana).
    - Desenvolvimento do microsserviço de **Pedidos** (responsável pela máquina de estados do pedido: RECEBIDO $\to$ EM_PREPARO $\to$ PRONTO).
    - Conexão dos microsserviços ao PostgreSQL com mapeamento ORM ou drivers nativos.

Fase 3: Camada de Tempo Real com Redis e Subscriptions

- **Foco:** Implementação da comunicação reativa bidirecional.
- **Entregas:**
    - Configuração do **Redis** como camada de mensageria (*Pub/Sub*).
    - Implementação de *Resolvers* do tipo Subscription para escutar eventos de novos pedidos e alterações de status.
    - Integração dos WebSockets no Apollo Server para suportar múltiplos clientes web conectados simultaneamente.

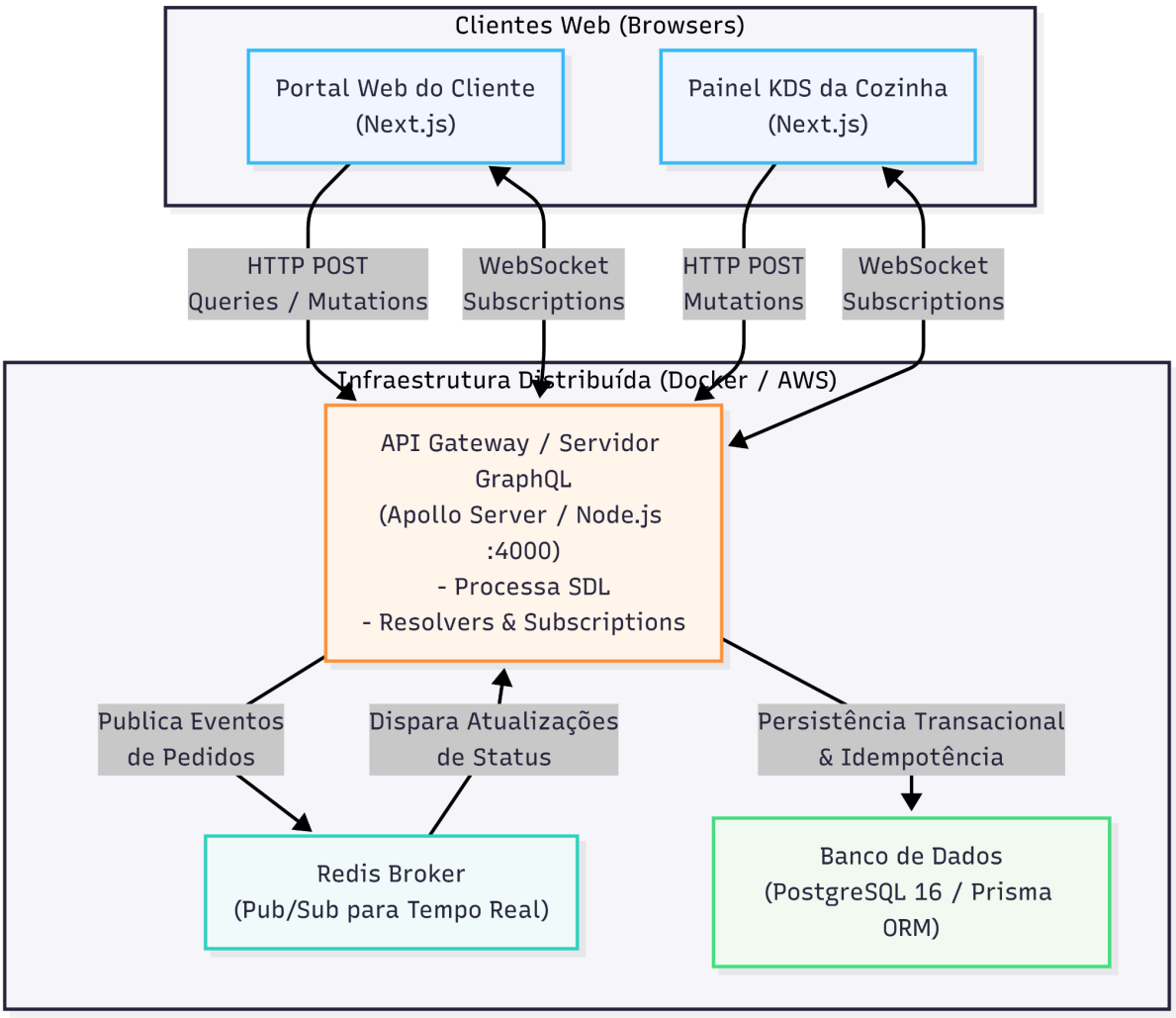
Fase 4: Testes de Carga, Concorrência e Resiliência

- **Foco:** Validação prática dos requisitos de sistemas distribuídos sobre o protocolo GraphQL.
- **Cenários Testados:**
    - **Cenário A (Alta Concorrência de Leitura):** Múltiplos navegadores consultando o cardápio simultaneamente com *queries* complexas e aninhadas.
    - **Cenário B (Queda do Broker / Redis):** Simulação de indisponibilidade do canal de tempo real e verificação de recuperação graciosa.
    - **Cenário C (Idempotência de Pedidos):** Disparo duplicado de *Mutations* de criação de pedidos para validar o tratamento de unicidade.

Fase 5: Integração das Interfaces Web e Orquestração

- **Foco:** Construção das telas de demonstração via navegador e empacotamento do sistema.
- **Entregas:**
    - Desenvolvimento das interfaces web em Next.js (Portal do Cliente e KDS da Cozinha).
    - Criação do arquivo docker-compose.yml para subir toda a infraestrutura (Servidor GraphQL, Microsserviços, Redis e PostgreSQL) com um único comando (docker compose up).

System Design do Projeto



Explicação do Fluxo de Arquitetura:

1. **Otimização de Tráfego (GraphQL):** O navegador web envia uma requisição enxuta especificando estritamente os campos necessários (ex: apenas o nome do produto e o preço, ignorando descrições longas), resolvendo o *over-fetching*.
2. **Resolução Distribuída:** O Apollo Server intercepta a requisição, analisa a árvore de execução (*AST*) e despacha as chamadas correspondentes para os microsserviços internos de Catálogo e Pedidos.
3. **Tempo Real com Subscriptions:** Quando a cozinha altera o status de um pedido através de uma *Mutation* no painel web, o microsserviço publica o evento no **Redis (Pub/Sub)**. O Redis despacha a mensagem para o servidor GraphQL, que por sua vez empurra a atualização via WebSocket para o navegador do cliente, garantindo baixa latência e sincronia em tempo real na lanchonete.